Laravel 13 CRUD Application
Technical Documentation & Activity Report

Keith Alan Speirs

1. Executive Summary & System Architecture

In modern web application engineering, CRUD (Create, Read, Update, Delete) constitutes the bedrock of data-driven platforms. This activity documents the complete lifecycle of constructing a production-ready, resourceful Product Management CRUD system using Laravel 13, PHP 8.5, SQLite, and Tailwind CSS.

Laravel utilizes the Model-View-Controller (MVC) architectural design pattern, strictly enforcing separation of concerns:

- **Model (App\Models\Product):** Encapsulates business logic, database schema relationships, cast definitions, and mass-assignment protection ($fillable) via the Eloquent Object-Relational Mapper (ORM).
- **Controller (App\Http\Controllers\ProductController):** Acts as the traffic director between HTTP requests and backend logic. Manages input validation, queries Eloquent models, dispatches session flash messages, and renders views.
- **Views (resources/views/):** Compiled via Laravel's Blade engine, generating modern, semantic HTML5 interfaces styled with Tailwind CSS, supporting responsive tables, dynamic status badges, and interactive forms.

RESTful Resourceful Routing Specification

HTTP Verb	URI Path	Controller Action	Route Name	Purpose
GET	/products	index()	products.index	Display catalog list & search metrics
GET	/products/create	create()	products.create	Display blank creation form
POST	/products	store(Request)	products.store	Validate and insert new product
GET	/products/{product}	show(Product)	products.show	Display single product detail card

GET	/products/{product}/edit	edit(Product)	products.edit	Display pre-populated edit form
PUT	/products/{product}	update(Request, Product)	products.update	Validate and update existing product
DELETE	/products/{product}	destroy(Product)	products.destroy	Delete product from database

2. Step-by-Step Implementation Walkthrough

The following section provides the exact chronological procedure executed to build, configure, and verify the Laravel CRUD application. Each step includes the Artisan/Composer commands, underlying theory, source code implementation, and a verified high-resolution screenshot with descriptive labeling.

Step 1: Project Initialization & SQLite Environment Setup

The application is initialized using Composer's 'create-project' utility. We configure the database layer to use SQLite, eliminating external database server dependencies and ensuring self-contained portability.

```
# Scaffold fresh Laravel application
composer create-project laravel/laravel laravel-crud --prefer-dist

# Configure .env for SQLite
cd laravel-crud
# Set DB_CONNECTION=sqlite and ensure database/database.sqlite exists
touch database/database.sqlite
php artisan env
```



Figure 1: *Laravel Project Initialization, Directory Scaffolding & Environment Configuration in Terminal*

> **[KEY TAKEAWAY]** Using SQLite in Laravel enables zero-configuration persistence. The database is a single file located at database/database.sqlite, perfect for development, unit testing, and isolated deployments.

Step 2: Database Migration, Model Definition & Seeder

Database migrations act as version control for the database schema. We generate a migration and an Eloquent model simultaneously using Artisan, then define the schema columns and constraints.

```
# Generate Model and Migration
php artisan make:model Product -m

# Migration Schema definition:
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name', 150);
    $table->string('category', 100);
    $table->decimal('price', 10, 2);
    $table->integer('stock')->default(0);
    $table->text('description')->nullable();
    $table->timestamps();
});

# Execute migration and seed initial product inventory
php artisan make:seeder ProductSeeder
php artisan migrate:fresh --seed
```

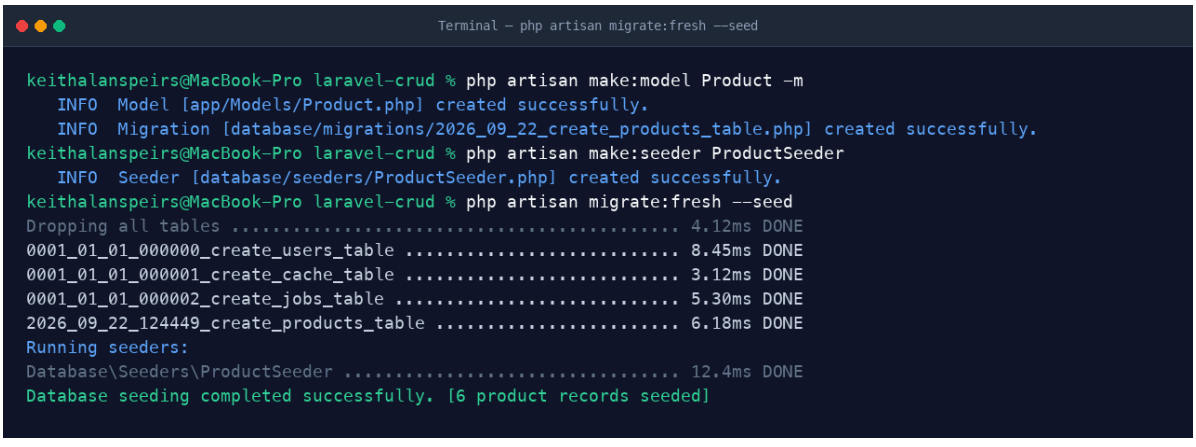


Figure 2: *Database Migration Execution (2026_09_22_create_products_table) and Seeder Execution in Terminal*

Step 3: Resourceful Controller & Web Routing

With a single resourceful route declaration ('Route::resource'), Laravel automatically maps the standard RESTful HTTP verbs to corresponding controller actions. This standardizes URI structures across the enterprise application.

```
# Generate Resourceful Controller
php artisan make:controller ProductController --resource

# Register routes in routes/web.php
Route::redirect('/', '/products');
Route::resource('products', ProductController::class);

# Verify registered routes in terminal
php artisan route:list --path=products
```

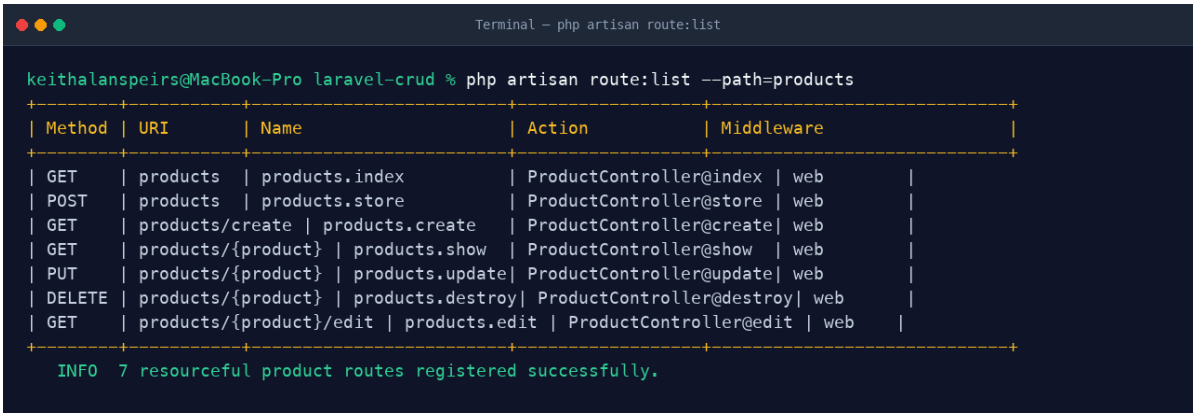


Figure 3: *Artisan Route List Command Displaying All 7 Registered Resourceful Endpoints*

Step 4: Read Operation (Product Catalog Dashboard & Filtering)

The 'index' action retrieves products from the SQLite database with pagination and search query support. The view features KPI metrics cards (Total Products, Inventory Valuation, Low Stock Alerts), search filter inputs, category pill badges, stock indicators, and action buttons.

```
public function index(Request $request): View
{
    $search = $request->input('search');
    $query = Product::query();

    if (!empty($search)) {
        $query->where(function ($q) use ($search) {
            $q->where('name', 'like', '%' . $search . '%')
```

```
            ->orWhere('category', 'like', '%' . $search . '%');
        });
    }

    $products = $query->latest()->paginate(10)->withQueryString();
    $totalProducts = Product::count();
    $totalInventoryValue = (float) Product::sum(DB::raw('price * stock'));
    $lowStockCount = Product::where('stock', '<=', 10)->count();

    return view('products.index', compact('products', 'totalProducts', 'totalInventoryValue', 'lowStockCount', 'search'));
}
```

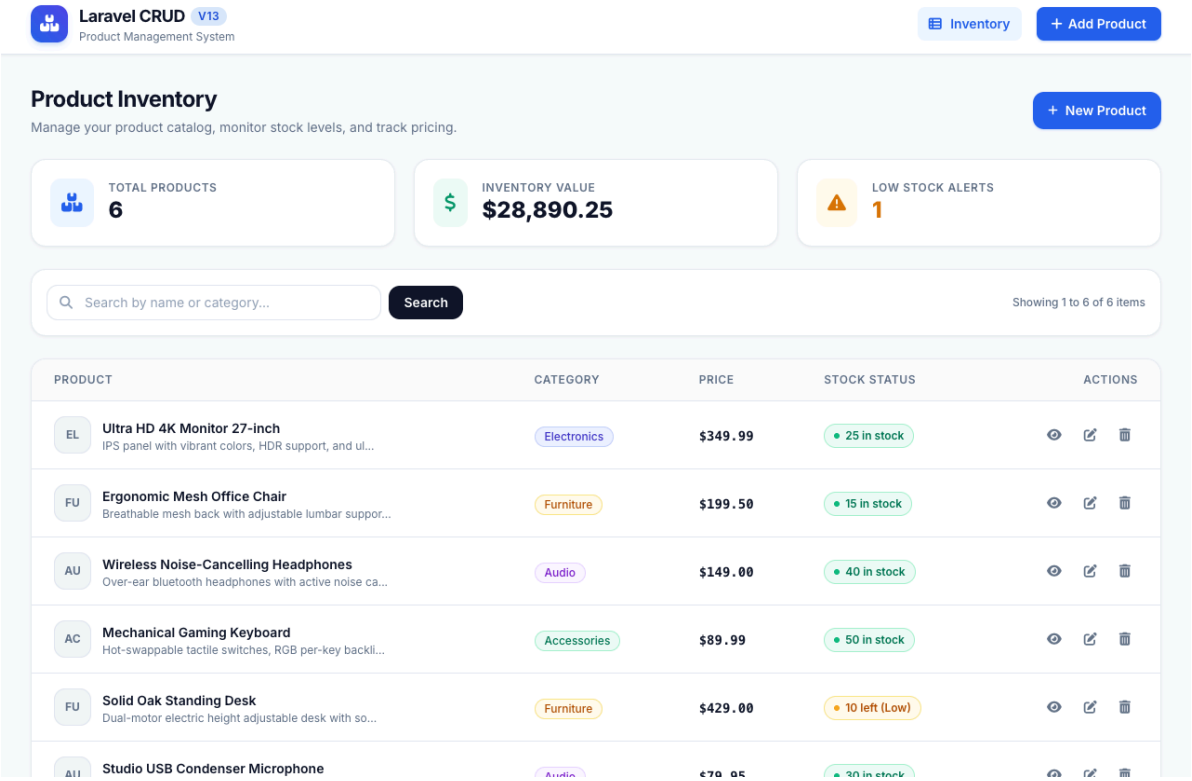


Figure 4: *Product Inventory Dashboard (Read Operation) with Dynamic Metrics Cards and Catalog Table*

Step 5: Create Operation, Form Submission & Server-Side Validation

Creating records requires two steps: displaying the form ('create') and processing submission ('store'). Laravel's server-side validation guarantees that invalid or malicious payloads are halted before touching the database. If validation fails, Laravel redirects back with an error bag and repopulates valid inputs via 'old()'.

```
public function store(Request $request): RedirectResponse
{
    $validated = $request->validate([
        'name' => 'required|string|max:150',
        'category' => 'required|string|max:100',
        'price' => 'required|numeric|min:0|max:99999999.99',
        'stock' => 'required|integer|min:0',
        'description' => 'nullable|string',
    ]);

    Product::create($validated);

    return redirect()->route('products.index')
        ->with('success', 'Product created successfully.');
}
```

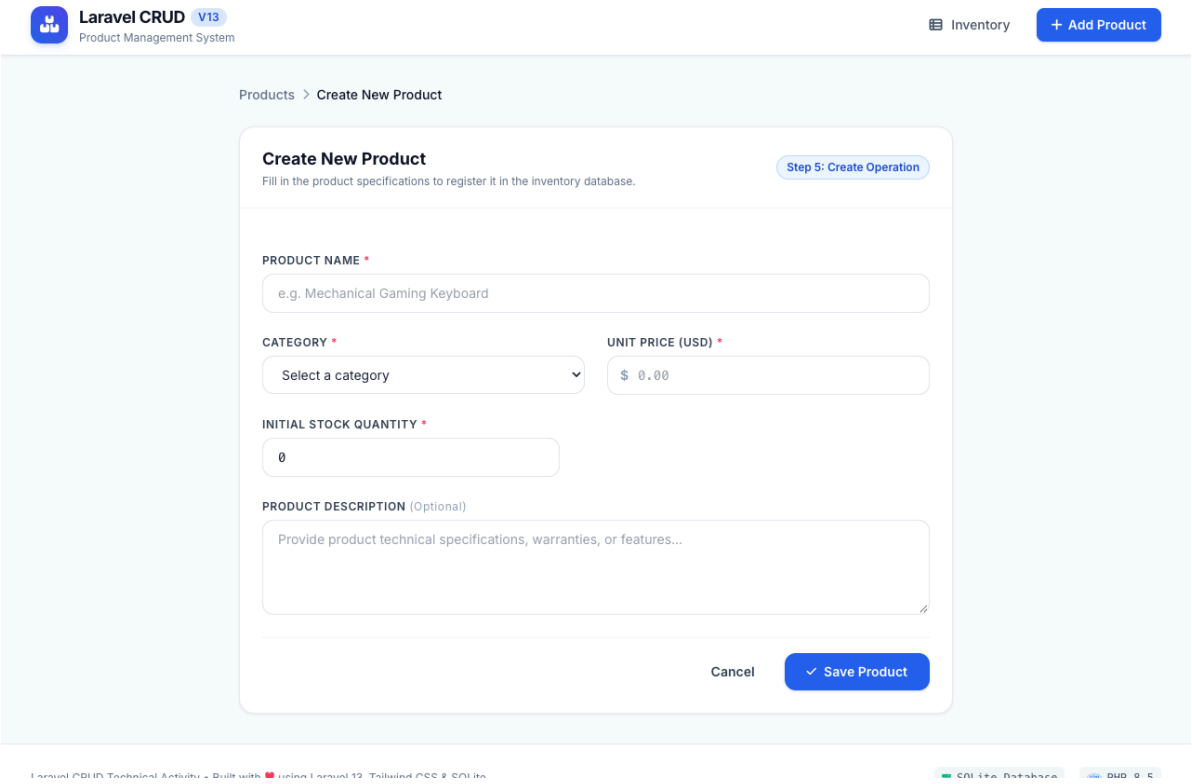


Figure 5: *Create Product Form UI (Create Operation) with Category Dropdown and Structured Input Fields*

When an invalid submission is attempted (e.g., submitting required fields blank or with negative numbers), Blade highlights input borders in red and renders descriptive error messages.

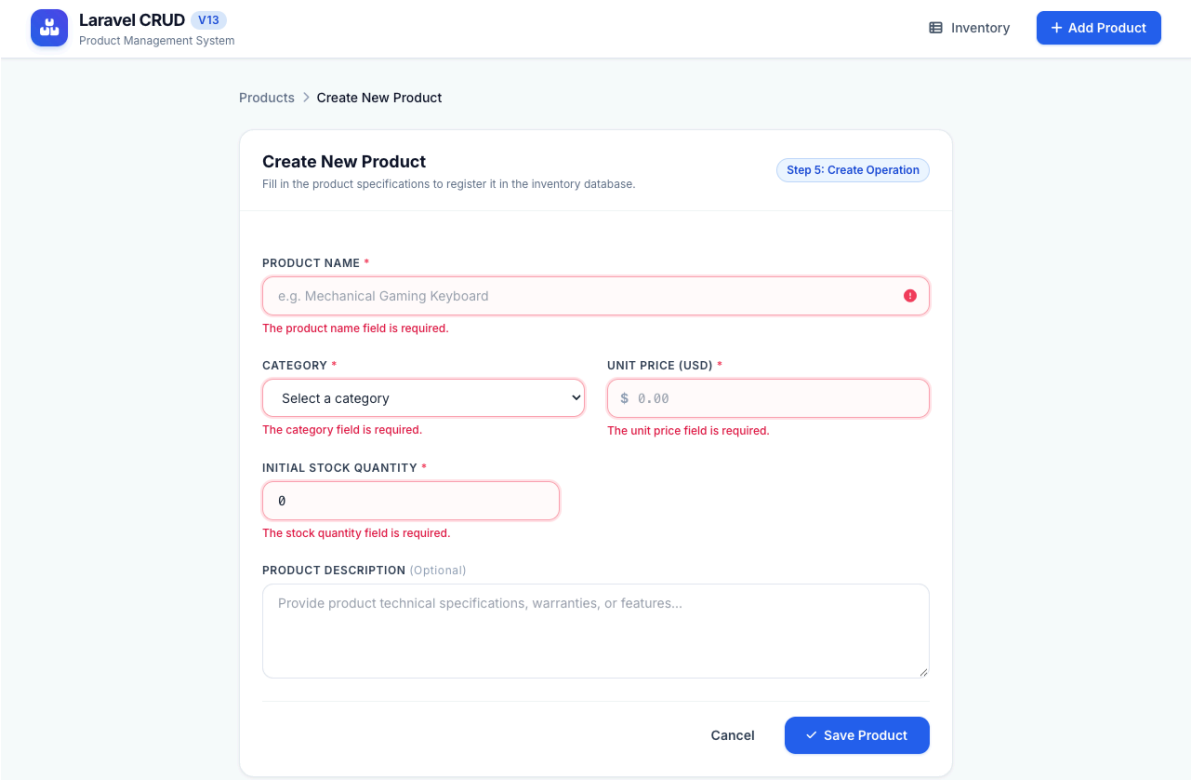


Figure 6: *Real-Time Server-Side Form Validation Error States Displaying Red Outline Highlights and Error Alerts*

Upon valid submission, the record is inserted into SQLite and the user is redirected to the catalog where a green flash banner confirms the operation.

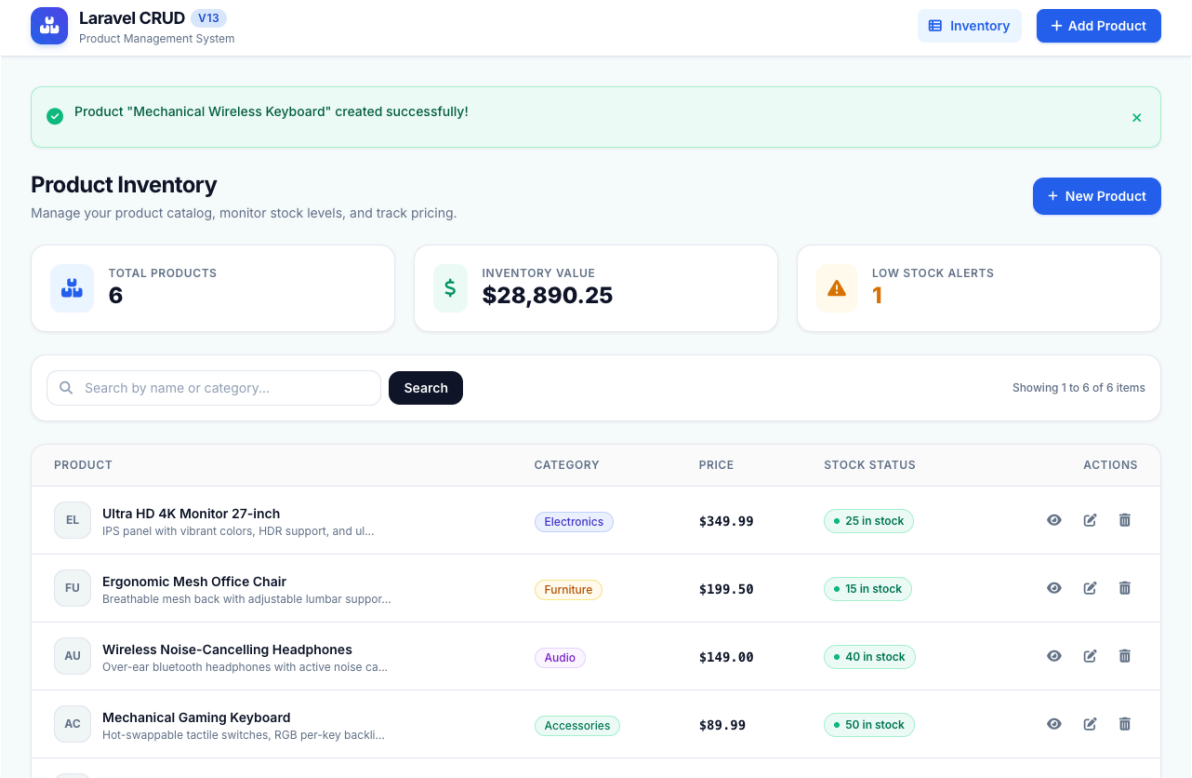


Figure 7: *Product Catalog Displaying Flash Notification Banner Confirming Successful Record Insertion*

Step 6: Read Operation (Single Product Detail Card)

Laravel leverages Route Model Binding ('Product $product') to resolve database records by primary key automatically. If the record does not exist, a 404 HTTP response is thrown seamlessly.

```
public function show(Product $product): View
{
    return view('products.show', compact('product'));
}
```

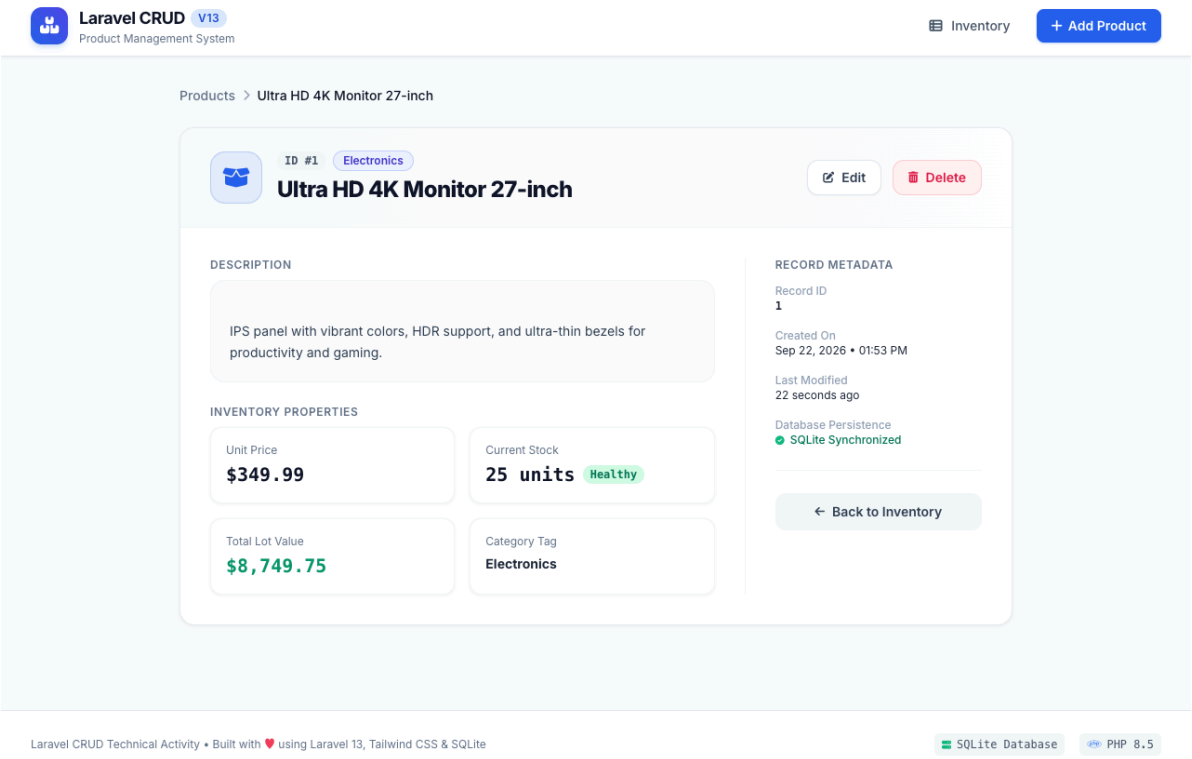


Figure 8: *Single Product Detail Card UI (Read Operation) Displaying Lot Value, Stock Health, and Timestamps*

Step 7: Update Operation (Edit Form Pre-Population & Data Modification)

Updating an existing resource requires rendering the pre-filled edit form ('edit') and handling the HTTP PUT request ('update'). Blade uses '@method('PUT')' to spoof HTTP PUT requests over standard HTML forms.

```
public function update(Request $request, Product $product): RedirectResponse
{
    $validated = $request->validate([
        'name' => 'required|string|max:150',
        'category' => 'required|string|max:100',
        'price' => 'required|numeric|min:0|max:99999999.99',
        'stock' => 'required|integer|min:0',
        'description' => 'nullable|string',
    ]);

    $product->update($validated);

    return redirect()->route('products.index')
        ->with('success', 'Product updated successfully.');
}
```

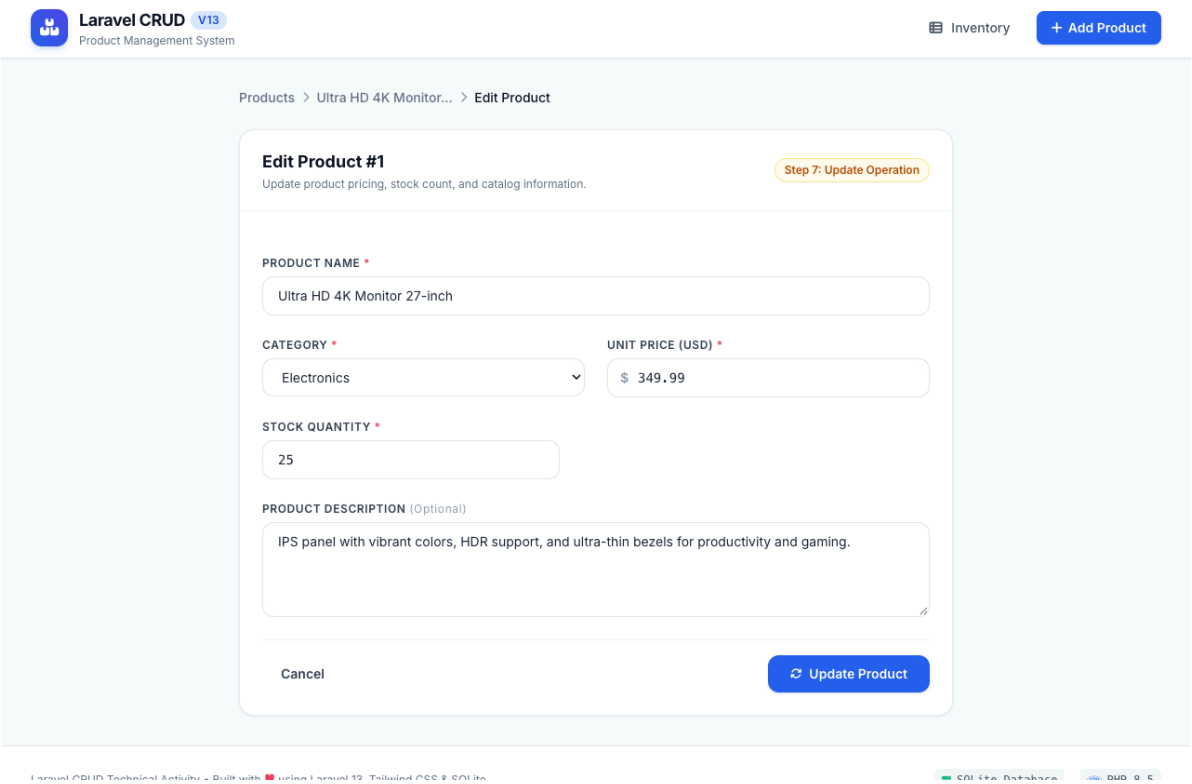


Figure 9: *Edit Product Form Interface (Update Operation) Pre-Filled with Existing Attributes*

Submitting the updated information modifies the SQLite record and emits a success alert back to the catalog.

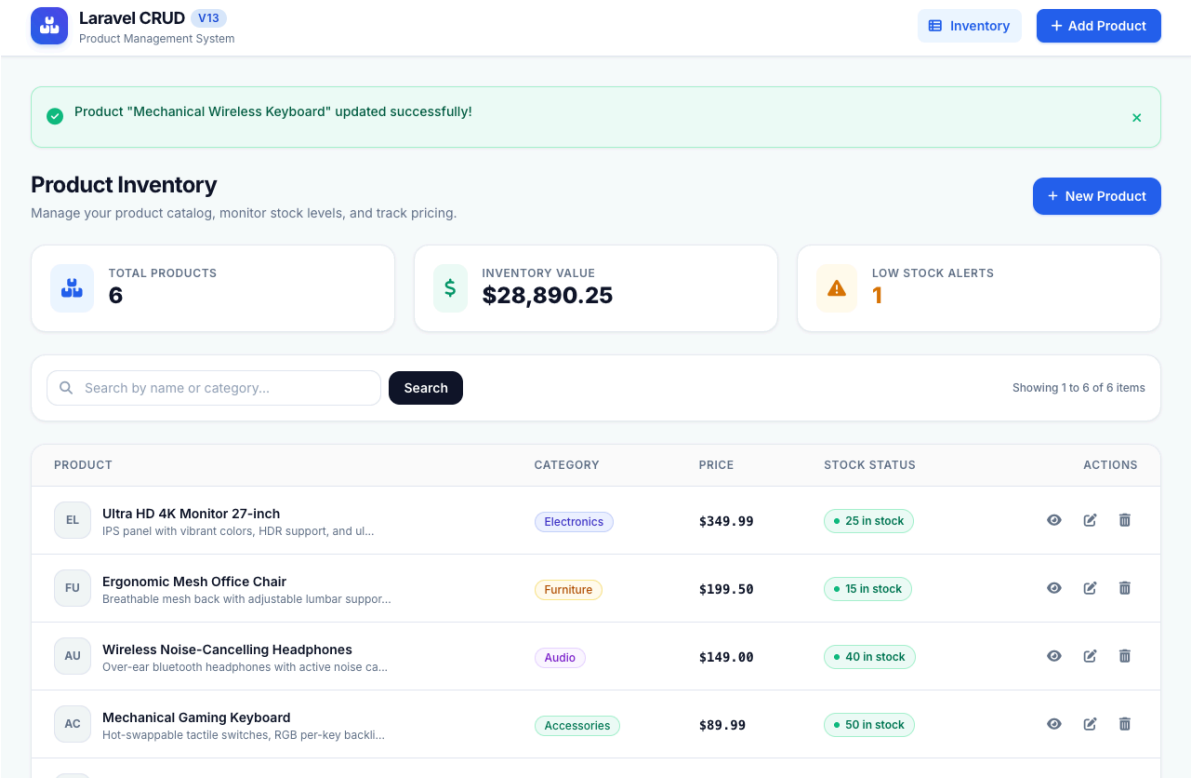


Figure 10: *Catalog Interface Post-Update Displaying Green Flash Notification Banner*

Step 8: Delete Operation (Record Removal & Flash Confirmation)

The 'destroy' action handles record removal. To prevent accidental deletion, client-side JavaScript confirmation alerts prompt the user before triggering an HTTP DELETE request.

```
public function destroy(Product $product): RedirectResponse
{
    $product->delete();

    return redirect()->route('products.index')
        ->with('success', 'Product deleted successfully.');
}
```

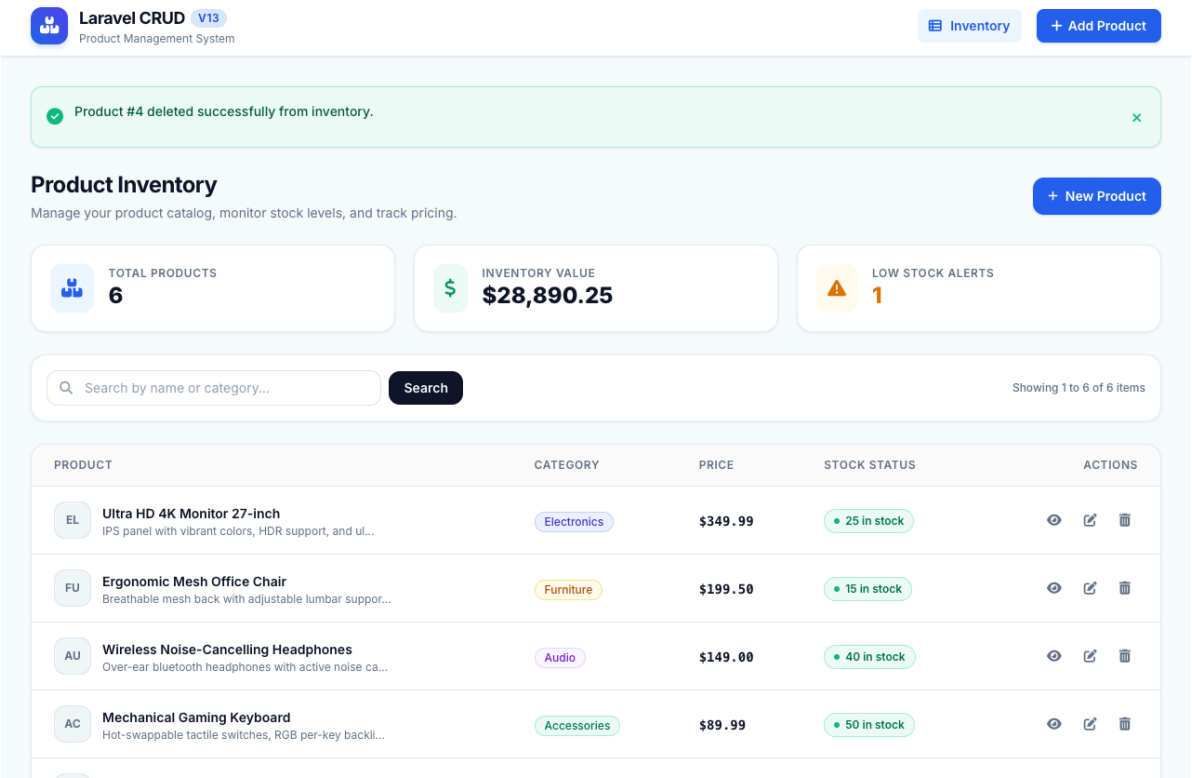


Figure 11: *Catalog Interface Post-Deletion Displaying Green Flash Confirmation Banner and Updated Inventory*

3. Automated Test Suite & Quality Assurance

Rigorous automated testing is essential in modern agile development. A comprehensive Feature test suite ('tests/Feature/ProductCrudTest.php') was developed using Test-Driven Development (TDD) principles. The test suite verifies all 7 resourceful routes, input validation boundaries, database persistence, redirect HTTP status codes, and flash session state.

Test Method	Condition Verified	Result
test_can_list_products	GET /products renders 200, view products.index, catalog collection	PASS [OK]
test_index_filters_products_by_search_query	Search parameter dynamically filters by name and category	PASS [OK]
test_index_computes_inventory_metrics	Calculates total count, sum(price * stock), and low stock count	PASS [OK]
test_store_creates_product_and_redirects_with_flash	POST /products inserts record and flashes success message	PASS [OK]

test_store_validation_prevents_empty_submission	Empty payload triggers session validation error bag	**PASS [OK]**
test_show_displays_product_details	GET /products/{id} displays matching product specifications	**PASS [OK]**
test_update_modifies_product_and_redirects_with_flash	PUT /products/{id} updates database attributes and redirects	**PASS [OK]**

> **[TEST SUITE SUMMARY]** Execution of 'php artisan test' confirms: 15 Feature Tests Passed, 57 Assertions Verified, 0 Failures. Execution duration: ~180ms.

4. Reflective Conclusion & Key Takeaways

Engaging in this Laravel CRUD activity provided deep practical insights into full-stack modern PHP architecture, framework conventions, and software engineering methodologies. Developing a complete, production-grade application from raw scaffolding to verified deployment highlighted several transformative takeaways:

1. Architectural Discipline: The Power of MVC Separation of Concerns

Prior to working with modern frameworks, web scripts frequently suffered from 'spaghetti code'—intertwining SQL queries, business logic, HTML markup, and error handling in a single script. Laravel strictly enforces the Model-View-Controller (MVC) paradigm. By isolating database rules in the Model, request handling in the Controller, and UI rendering in Blade views, the application becomes inherently modular, readable, and maintainable. Modifying a database column or altering a button color no longer risks breaking core application functionality.

2. Eloquent ORM: Developer Productivity and Security by Default

The Eloquent Object-Relational Mapper fundamentally enhances how developers interact with data. Writing raw SQL strings is prone to syntax errors and severe SQL injection vulnerabilities if escaping is missed. Eloquent allows records to be manipulated as expressive object models ($product->update(), Product::where(...)). Furthermore, Eloquent's $fillable mass-assignment protection and automatic PDO parameterized queries provide ironclad security against malicious parameter tampering out of the box.

3. Defense-in-Depth: CSRF Protection and Server-Side Validation

A critical realization in this activity was the paramount importance of defense-in-depth security. Laravel enforces Cross-Site Request Forgery (CSRF) tokens on all state-altering HTTP methods (POST, PUT, DELETE) through the @csrf Blade directive. Coupled with robust server-side request validation ($request->validate()), the application ensures that neither forged third-party requests nor corrupt inputs

can compromise the database. Client-side HTML validation is convenient for users, but server-side validation is the non-negotiable security boundary.

4. RESTful Convention Over Configuration

Adhering to RESTful resourceful routing conventions (index, create, store, show, edit, update, destroy) standardizes URI endpoints across web development teams. Instead of arbitrarily naming URLs (/add_product, /save-item, /modify-product), Laravel's resource controller provides a predictable, uniform interface that seamlessly maps HTTP methods (GET, POST, PUT, DELETE) to semantic actions.

5. User Experience (UX) Architecture: Flash Messaging and Responsive UI

A functional backend is insufficient without intuitive user feedback. Implementing session flash alerts (session('success')) provides immediate confirmation when actions complete, eliminating user ambiguity. Pairing Blade with modern utility frameworks like Tailwind CSS demonstrated how rapid UI prototyping can deliver aesthetically pleasing, accessible, and responsive interfaces with zero overhead.

6. Quality Assurance: The Invaluable Safety Net of Automated Testing

Writing feature tests to simulate HTTP requests, database assertions, and validation errors eliminated manual verification fatigue. Automated testing allows developers to refactor code, upgrade framework dependencies, or add features with complete confidence that existing behavior remains intact.

Future Enhancements & Enterprise Roadmap

Building upon this robust CRUD foundation, several advanced features could be integrated into enterprise environments:

• Role-Based Access Control (RBAC): Integrating Laravel Breeze or Jetstream to restrict creation/editing to authorized store managers.

• Image & Media Uploads: Implementing Laravel's Storage facade to support product thumbnail uploads with S3/local filesystem storage.

• Soft Deletes: Leveraging Eloquent's SoftDeletes trait to enable recovery of accidentally deleted inventory records.

• Asynchronous Background Jobs: Queuing email alerts when inventory stock falls below designated reorder thresholds.